

CortexWeave: Systems Architecture for Small-Model Agentic intelligence

Small Models, Smart Systems

A Roadmap Toward Frontier-Level Agentic Results

1. Vision

The central goal of this project is to explore a different path toward frontier-level agentic capability.

Instead of relying primarily on increasingly large language models, the system attempts to achieve comparable **task-level outcomes** using:

- small local language models
- deterministic systems
- persistent external memory
- semantic retrieval
- explicit state management
- compilers, test suites, and other verifiers
- structured tool use
- iterative feedback loops
- code and dependency analysis
- intelligent context construction
- accumulated experience across sessions

The objective is not necessarily to make a 2B or 4B model reason identically to a frontier model.

The objective is:

Given the same task, can a small-model system eventually produce an equally correct result, even if it requires more steps and uses fundamentally different methods?

This shifts the optimization target away from **single-inference intelligence** and toward **whole-system capability**.

A useful conceptual model is:

```
capability ≠ model size alone
```

```
capability ≈
```

```
model intelligence
```

```
× retrieval quality
```

```
× state quality
```

```
× tool quality
```

```
× verification quality
```

```
× iteration count
```

A frontier model may solve a problem in one or two inference passes.

A small-model system might instead perform:

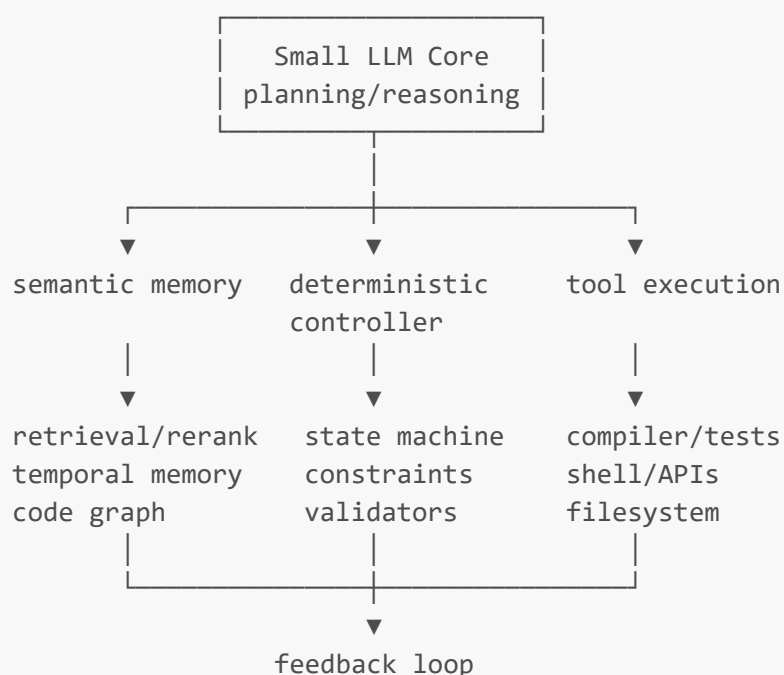
```
generate
↓
verify
↓
observe failure
↓
retrieve relevant knowledge
↓
diagnose
↓
repair
↓
verify again
↓
repeat until correct
```

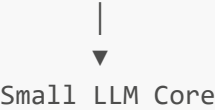
If inference is inexpensive enough and the feedback loop is reliable, requiring more iterations is not necessarily a disadvantage.

The target metric becomes **eventual correctness per task**, rather than the quality of a single inference.

2. Core Design Principle

The language model should be treated as **one component of a larger cognitive system**, not as the entire cognitive system.





Every weakness of the model should prompt a systems question:

Can this responsibility be moved outside inference?

Examples:

Model weakness	External solution
Poor memory	Persistent semantic memory
Type mistakes	Compiler
Hallucinated files	Filesystem
Lost task state	Explicit state machine
Forgotten decisions	Decision retrieval
Weak exact search	Lexical search / FTS
Poor code navigation	Tree-sitter / LSP / code graph
Context overflow	Context assembly
Repeated mistakes	Failure/fix memory
Uncertain correctness	Tests and validators
Weak long-horizon reasoning	Iterative planning and execution
Context pollution	Working-set management

A fundamental rule follows:

The model should never be the source of truth when a deterministic source of truth exists.

Types

Files

Git state

Tests

Dependencies

Symbols

Memory

Task state

→ compiler

→ filesystem

→ Git

→ test runner

→ package manager

→ parser / LSP

→ persistent store

→ orchestrator

The model interprets these systems.

It does not invent their state.

3. Context Should Be Constructed, Not Accumulated

Large context windows make it tempting to continuously append conversation history.

That eventually produces:

```
64K tokens of conversation
old errors
obsolete plans
superseded code
irrelevant discussion
duplicated information
    ↓
model
```

Instead, context should be dynamically constructed.

```
current task
+
relevant code
+
active decisions
+
recent failures
+
important historical knowledge
+
current repository state
    ↓
compact high-quality context
    ↓
small model
```

A carefully selected 8K-token context can potentially be more useful than an indiscriminately accumulated 64K-token transcript.

This requires an external memory system capable of deciding **what deserves to return to model context**.

4. The Semantic Memory Project

The foundational component is a local semantic-memory service for coding agents.

Its purpose is not simply to provide vector search.

Its purpose is:

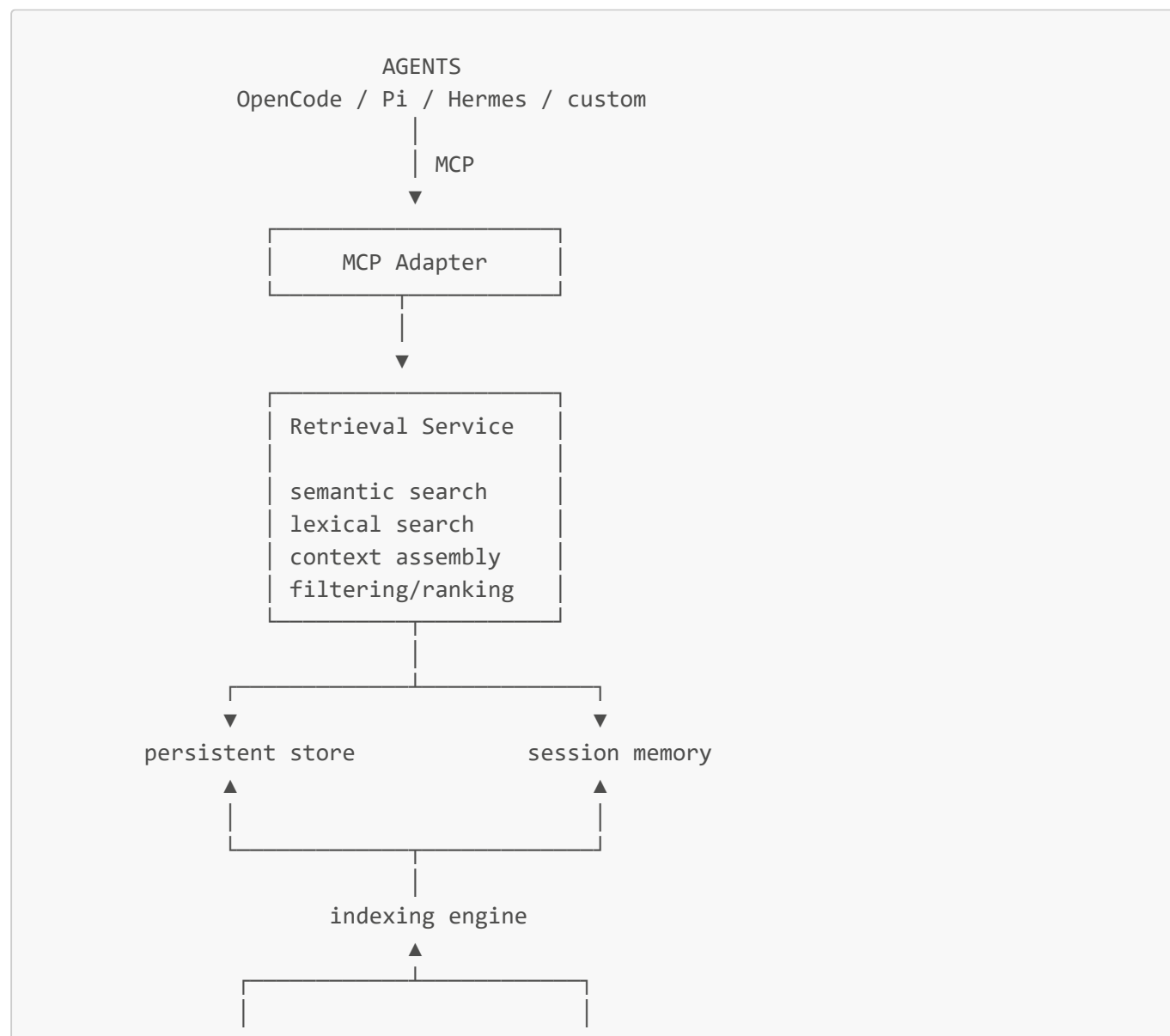
Provide persistent, near-realtime, structured working memory for local development agents.

The service should:

- watch project directories continuously
- incrementally index code as it changes
- understand code structure
- persist semantic representations
- index important session knowledge
- expose retrieval through MCP
- construct useful context within token budgets
- preserve knowledge across model context resets
- track how knowledge changes over time
- eventually learn from previous failures and successful solutions

The semantic system becomes an **external hippocampus** for the agent.

5. High-Level Architecture





The MCP interface should remain a thin adapter.

The semantic engine should not depend on MCP internally.

This allows the same core to support:

- MCP
- CLI
- HTTP
- custom agent frameworks
- IDE integrations
- direct Rust library integration
- future protocols

6. Phase 1: Semantic Core

Build the smallest reliable semantic engine first.

Do not begin with MCP.

Do not begin with filesystem watching.

Start with:

```
graph TD; A[text] --> B[embedding client]; B --> C[persistent semantic index];
```

The diagram shows a simple three-step process. It starts with 'text', which goes down to an 'embedding client', which then goes down to a 'persistent semantic index'. The steps are connected by downward-pointing arrows.



search

Components

Embedding Client

Use an OpenAI-compatible embedding API.

Initial target:

```
http://localhost:8081/v1/embeddings
```

Responsibilities:

- configurable endpoint
- configurable model
- batch embedding
- timeout handling
- retry handling
- dimension validation
- backend health checks

The Rust process should not initially load an embedding model itself.

The embedding server owns inference.

Persistent Storage

Use SQLite initially.

Store concepts such as:

```
workspaces
documents
chunks
embeddings
metadata
```

Example logical schema:

```
documents
  id
  workspace_id
  path
  content_hash
  modified_at
```

```
chunks
  id
  document_id
  stable_key
  text
  content_hash
  language
  symbol
  start_line
  end_line
  indexed_at

embeddings
  chunk_id
  model
  dimension
  vector
```

Semantic Search

Initially support:

- add
- upsert
- delete
- cosine similarity
- top-k retrieval
- embedding dimension enforcement

Exit Criterion

Data can be manually indexed, the service can restart, and that data remains semantically searchable.

7. Phase 2: Workspace Ingestion

Teach the service how to understand a repository.

Input:

```
/path/to/project
```

Output:

```
persistent searchable semantic representation
```


Implement:

- recursive traversal
- `.gitignore` support
- configurable include/exclude patterns
- binary detection
- file-size limits
- language detection
- document hashing
- initial indexing
- deletion detection
- stable workspace identity

Default exclusions might include:

```
.git/  
target/  
node_modules/  
dist/  
build/  
binary assets  
generated files
```

Each workspace should receive a stable identity.

```
workspace  
├── UUID  
├── root path  
└── repository metadata
```

Exit Criterion

Point the service at a real repository and obtain useful semantic results.

8. Phase 3: Structure-Aware Chunking

Fixed-size text chunks are insufficient for high-quality code retrieval.

For supported programming languages, use Tree-sitter.

```
source file  
  ↓  
Tree-sitter  
  ↓  
functions
```

```
methods
classes
structs
impl blocks
modules
```

For unsupported languages and prose:

```
document structure
  ↓
headings / paragraphs
  ↓
bounded token chunks
  ↓
small overlap
```

A chunk should contain structural metadata:

```
struct Chunk {
  id: Uuid,
  stable_key: String,
  path: PathBuf,

  language: Option<String>,
  symbol: Option<String>,

  start_line: usize,
  end_line: usize,

  text: String,
  content_hash: String,
}
```

Stable Chunk Identity

Chunk identity should not depend entirely on its current text.

A stable key could derive from:

```
workspace_id
+
relative_path
+
symbol
+
structural anchor
```

Then:

```
same function
+
different implementation
=
same semantic object
+
new content version
```

This enables precise replacement rather than accumulating obsolete vectors.

Exit Criterion

Modify one function and demonstrate that only the affected chunks require re-embedding.

9. Phase 4: Near-Realtime Incremental Indexing

Add filesystem monitoring using a Rust watcher such as [notify](#).

Pipeline:

```
filesystem event
  ↓
debounce
  ↓
change aggregation
  ↓
content hash
  ↓
parse changed file
  ↓
chunk diff
  ↓
embedding queue
  ↓
upsert/delete
```

A debounce period around 200 to 500 ms is a reasonable starting point.

Editors frequently generate multiple filesystem events for one logical save.

Therefore:

```
file changed
file changed
```

```
file changed
formatter changed file
    ↓
one indexing operation
```

Chunk-Level Invalidation

Compare previous and current chunks.

```
same stable key + same hash
    → nothing

same stable key + new hash
    → re-embed

new stable key
    → embed

missing old stable key
    → delete
```

This makes indexing proportional to **actual semantic change**, rather than repository size.

Exit Criterion

Save a source file and have the new implementation become searchable within approximately a second to a few seconds.

10. Phase 5: MCP Interface

Once the core works reliably, expose it through MCP.

Initial tools:

```
semantic_search
semantic_get
semantic_status
semantic_reindex
```

Example query:

```
{
  "query": "where is request authentication handled?",
```

```
"k": 8  
}
```

Result metadata should include:

```
score  
workspace  
path  
symbol  
line range  
text  
source type
```

Later filters can include:

```
workspace  
path  
language  
symbol  
source_type  
time range
```

Exit Criterion

An MCP-capable coding agent can query the live repository index.

11. Phase 6: Session Memory

Code is only one form of useful memory.

Add session artifacts such as:

```
code  
documentation  
decision  
agent_note  
task  
git_diff  
terminal_output  
compiler_error  
test_result  
tool_result  
conversation_summary
```

Example memory:

```
type: decision

decision:
Use SQLite for initial persistence.

reason:
Keep deployment local and dependency-light.
```

Later:

```
"Why did we choose SQLite?"
```

can retrieve the actual decision.

Avoid Transcript Dumping

Do not blindly embed every conversation token.

Prefer high-value information:

- architectural decisions
- discoveries
- hypotheses
- failures
- fixes
- TODOs
- implementation changes
- test results
- important commands
- checkpoints

The memory system should preserve **knowledge**, not merely history.

Exit Criterion

A fresh agent session can retrieve important decisions from a previous session.

12. Phase 7: Explicit Memory Operations

Agents should be able to intentionally store knowledge.

Potential MCP tools:

```
memory_record  
memory_checkpoint  
memory_search  
memory_recent
```

Structured memory categories might include:

```
decision  
failure  
solution  
hypothesis  
todo  
observation  
constraint
```

For example:

```
memory_record(  
    type = "decision",  
    text = "...",  
    related_files = [...]  
)
```

Checkpoints

`memory_checkpoint` should create a compact representation of:

```
current objective  
what changed  
what was decided  
important failures  
important discoveries  
relevant files  
remaining work
```

This enables session continuation without retaining an enormous conversation transcript.

13. Automatic Session Resurrection

A new model context should be able to request:

```
resume_context(workspace="/project")
```

The memory system could construct:

```
Current task:
Implement near-realtime incremental indexing.

Recent decisions:
- use notify
- debounce for 250 ms
- Tree-sitter chunks use stable symbol identities
- embeddings come from localhost:8081

Recent failures:
- rename events create duplicate chunks

Relevant files:
- src/watcher.rs
- src/indexer.rs
- src/chunks.rs

Git state:
3 modified files
branch: feature/live-index

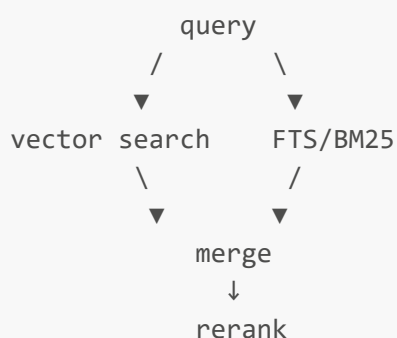
Likely next action:
Fix rename reconciliation and rerun watcher tests.
```

This allows small models to recover sophisticated working state without carrying the original conversation.

14. Phase 8: Hybrid Retrieval

Embeddings alone are not sufficient for code.

Combine semantic and lexical retrieval.



SQLite FTS5 is a strong initial lexical backend.

Lexical retrieval handles identifiers such as:

```
EmbeddingClient  
handle_search  
E0425  
SemanticIndex
```

Semantic retrieval handles concepts such as:

```
where do we reject invalid vector dimensions?
```

A combined score might eventually resemble:

```
final_score =  
    semantic_weight * semantic_score  
+ lexical_weight   * lexical_score  
+ recency_weight   * recency_score
```

15. Phase 9: Context Assembly

Retrieval results should eventually be transformed into coherent **context packages**.

Pipeline:

```
query  
↓  
retrieve candidates  
↓  
deduplicate  
↓  
structural expansion  
↓  
rerank  
↓  
token-budget optimization  
↓  
context packet
```

Suppose retrieval finds:

```
src/index.rs
SemanticIndex::search
lines 104-148
```

The assembler may include:

- the function
- its relevant type definition
- necessary neighboring code
- related decision memory
- a recent failure involving that function

while remaining inside a token budget.

Expose:

```
semantic_context
```

with arguments such as:

```
query
token_budget
include_code
include_docs
include_session_memory
```

The model receives **usable context**, rather than a raw bag of search results.

16. Phase 10: Temporal Memory

Do not treat knowledge as timeless.

Preserve versions.

The system should understand:

```
what exists now
what existed previously
what changed
when it changed
why it changed
```

This enables queries such as:

```
What changed in the indexing system today?  
  
What did this function look like before the refactor?  
  
When did we stop using L2 distance?  
  
Which decision superseded the previous storage design?
```

Memory becomes a timeline rather than a static vector collection.

17. Phase 11: Event-Sourced Session State

Represent meaningful development activity as events.

Examples:

```
FileModified  
CompilerFailed  
TestFailed  
TestPassed  
DecisionRecorded  
PatchApplied  
CommitCreated  
TaskStarted  
TaskCompleted  
HypothesisRejected
```

This creates relationships such as:

```
changed X  
  ↓  
cargo check failed with Y  
  ↓  
changed Z  
  ↓  
tests passed
```

The system can begin retrieving not only information, but **experience**.

18. Phase 12: Memory Consolidation

Raw events should not remain the primary retrieval unit forever.

Periodically consolidate them.

For example:

```
compiler error
compiler error
patch
compiler error
patch
tests pass
```

may consolidate into:

```
Problem:
Embedding IDs were represented inconsistently.

Resolution:
Use UUID consistently across SemanticIndex APIs.

Verification:
cargo check and tests passed.
```

The original events can remain available, but the consolidated memory becomes the preferred long-term representation.

This creates multiple memory layers:

```
raw events
  ↓
episodic memory
  ↓
consolidated knowledge
  ↓
long-term semantic memory
```

19. Phase 13: Importance and Forgetting

Not every memory deserves permanent prominence.

Assign importance using factors such as:

```
recency
retrieval frequency
explicit importance
source reliability
task relevance
supersession
successful reuse
```

Low-value memories can eventually be:

- deprioritized
- consolidated
- archived
- deleted

This provides a form of **memory garbage collection**.

20. Phase 14: Provenance and Confidence

Every memory should record where it came from.

Examples:

```
current source code
compiler output
test output
Git
user instruction
agent inference
agent summary
documentation
```

These sources should not be treated equally.

For example:

```
compiler result
  >
current source
  >
old agent speculation
```

for questions about whether code compiles.

Provenance allows retrieval and context assembly to prefer authoritative evidence.

21. Phase 15: Contradiction and Supersession Detection

Suppose memory contains:

```
Decision:  
Use L2 distance.
```

but the current implementation and a later decision use cosine similarity.

The system should not return both as equally valid.

Instead:

```
L2 decision  
status: superseded  
  
Cosine decision  
status: active
```

This prevents obsolete semantic ghosts from polluting context.

22. Phase 16: Code Graph

Pair semantic retrieval with structural understanding.

Build relationships such as:

```
function → callers  
function → callees  
type → implementations  
module → imports  
symbol → tests  
file → dependencies
```

Then retrieval can combine:

```
semantic relevance  
+  
lexical relevance
```

```
+  
structural relevance
```

If semantic search finds:

```
SemanticIndex::search
```

the system can automatically identify:

- callers
- related tests
- dependent types
- relevant traits
- neighboring implementation code

This is substantially more powerful than vector retrieval alone.

23. Phase 17: Change-Impact Analysis

Once a code graph exists, filesystem changes can propagate potential impact.

```
function changed  
  ↓  
direct callers  
  ↓  
dependent modules  
  ↓  
related tests
```

The agent can ask:

```
What might this edit have broken?
```

and receive structurally informed candidates.

24. Phase 18: Predictive Context Prefetching

The system can eventually predict likely retrieval needs.

If the agent begins editing:

```
src/indexer.rs
```

the system can prefetch:

```
related tests
recent decisions
previous failures
called functions
dependent modules
recent Git history
```

before the agent requests them.

This turns retrieval latency into background work.

25. Phase 19: Working-Set Memory

Separate long-term memory from what currently matters.

```
long-term memory
  ↓
dynamic working set
  ↓
assembled context
  ↓
small model
```

The working set might contain:

- current task
- active files
- recent decisions
- unresolved failures
- current hypotheses
- likely dependencies

As the task changes, the working set changes.

This gives the system an external approximation of **attention across time**.

26. Phase 20: Query Decomposition

Complex questions should not necessarily become one embedding query.

Example:

```
Why is incremental indexing missing edits after Git checkout?
```

could decompose into:

```
filesystem watcher behavior  
Git checkout handling  
debounce logic  
previous indexing failures  
recent changes to watcher code
```

Each query can be retrieved independently and the results fused.

A very small local model may be sufficient for this role.

27. Phase 21: Local Reranking

Embedding similarity generates candidates.

A small secondary model can determine which candidates actually answer the question.

```
semantic retrieval  
  ↓  
20 candidates  
  ↓  
0.5B-2B reranker  
  ↓  
5 useful candidates  
  ↓  
context assembler
```

This is a natural role for extremely small models because the task is narrow and bounded.

28. Phase 22: Git Awareness

Track:

```
branch  
commit
```

```
dirty state
changed files
diffs
renames
```

This enables:

```
What changed in the routing system this session?

What did this branch change about indexing?

Where did we modify embedding batching?
```

Git awareness also improves filesystem reconciliation.

A checkout may produce an enormous filesystem event storm.

Instead of blindly processing everything:

```
Git event
  ↓
determine actual content changes
  ↓
reconcile only affected semantic objects
```

29. Phase 23: Branch-Aware Memory

Memory should understand alternate Git histories.

A decision made on:

```
feature/sqlite
```

may not apply to:

```
main
```

Memories should therefore carry branch/revision context where appropriate.

This prevents knowledge from alternate development timelines leaking into current reasoning.

30. Phase 24: Task-Scoped Memory

Introduce explicit tasks.

Example:

```
task:
  fix incremental indexing
```

Automatically associate:

```
searches
files
decisions
errors
patches
tests
notes
```

with that task.

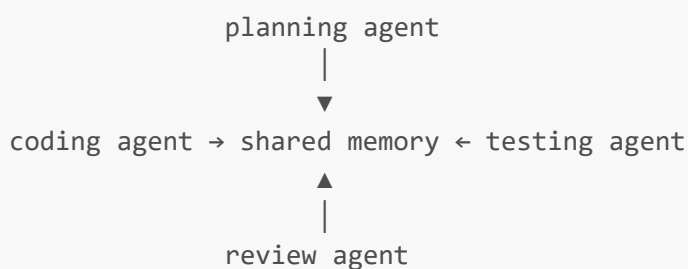
Then:

```
resume_task("fix incremental indexing")
```

can reconstruct the relevant working state.

31. Phase 25: Multi-Agent Shared Memory

Multiple specialized agents can use the semantic system as a shared blackboard.



Agents can contribute:

```
discoveries
failures
decisions
test results
implementation notes
```

without exchanging entire transcripts.

This enables specialization without catastrophic context duplication.

32. Phase 26: Embedding Backend Abstraction

The embedding backend should be replaceable.

Conceptually:

```
trait EmbeddingProvider {
  async fn embed(
    &self,
    inputs: &[String],
  ) -> Result<Vec<Vec<f32>>>;
}
```

Initial implementation:

```
OpenAI-compatible HTTP
```

Future implementations could support:

```
llama.cpp
Ollama
LM Studio
remote APIs
embedded inference
```

Store:

```
embedding model
embedding dimension
embedding version
```

with the index.

Never silently mix vectors from incompatible embedding spaces.

Changing models should trigger controlled migration or reindexing.

33. Phase 27: Resource Scheduling

Local inference resources are constrained.

The system should understand priority.

```
HIGH
interactive semantic query

MEDIUM
currently edited file

LOW
background repository indexing
```

Configuration should include:

```
embedding batch size
maximum concurrency
debounce duration
queue size
background indexing rate
resource limits
```

An agent waiting for retrieval should not be blocked because the indexer decided to embed 4,000 documentation chunks.

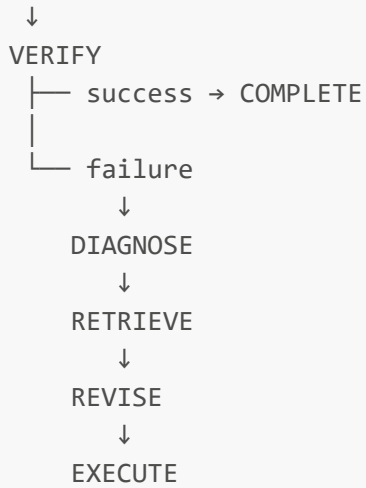
34. Phase 28: Deterministic Controller

As the system grows, orchestration should not itself become an unconstrained LLM prompt.

Use deterministic state machines where possible.

Example task lifecycle:

```
PLAN
↓
EXECUTE
```



The model operates **inside states**.

The deterministic controller decides what transitions are legal.

This reduces wandering and preserves long-horizon task state.

35. Phase 29: Verification as External Cognition

Small models should aggressively exploit deterministic feedback.

For Rust:

```
generate patch
  ↓
cargo check
  ↓
compiler diagnostics
  ↓
model diagnoses
  ↓
repair
  ↓
cargo check
  ↓
cargo test
```

The compiler effectively becomes an external type-reasoning system.

Similar patterns apply elsewhere:

```
Python → tests / type checker / linter
Web → browser / DOM / screenshot validation
SQL → database
```

```
Git → repository state
APIs → schemas and responses
```

The environment becomes part of the reasoning process.

36. Failure as Learning Data

Failures should not simply disappear after repair.

Store relationships such as:

```
failure
  ↓
diagnosis
  ↓
attempted fix
  ↓
verification result
```

Example:

```
Failure:
Rust type mismatch between semantic ID and storage ID.

Cause:
Uuid used in one layer and u64 in another.

Resolution:
Standardize SemanticIndex APIs on Uuid.

Verification:
cargo check succeeded.
```

If the system encounters a similar problem later, semantic retrieval can surface the previous solution.

This creates **experience without modifying model weights**.

37. Capability Accumulation Without Training

Over time, persist:

```
successful patches
failure → fix pairs
```

```
architecture decisions
project conventions
tool-use strategies
known bad approaches
test outcomes
retrieval traces
task outcomes
```

Therefore:

```
fixed model weights
      +
growing external experience
      +
better retrieval
      +
better orchestration
      =
increasing system capability
```

The system can improve without fine-tuning the underlying model.

38. Three Timescales of Cognition

The architecture naturally separates cognition into different speeds.

Fast

Mostly deterministic:

```
routing
validation
symbol lookup
filesystem queries
cache access
state transitions
```

Medium

Small-model inference:

```
reasoning
generation
classification
```



```
tool selection  
local planning  
query decomposition
```

Slow

Iterative processes:

```
multi-pass investigation  
compiler/test loops  
repository analysis  
memory consolidation  
cross-session learning  
long-horizon tasks
```

Frontier models internalize much of this work inside very large inference processes.

This architecture deliberately decomposes it into specialized systems.

39. Multi-Model Architecture

A possible mature system could use several small models.

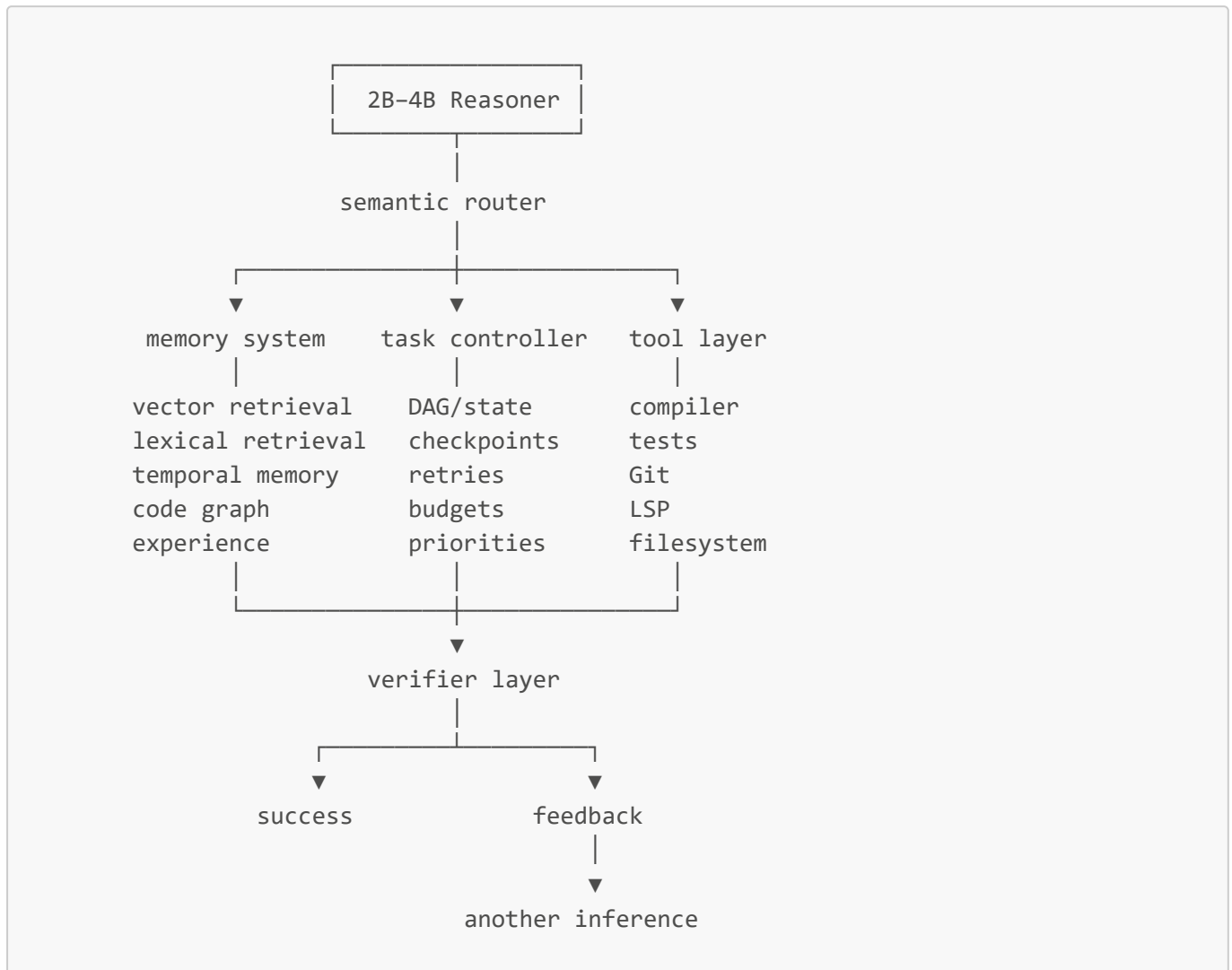
```
0.5B-1B model  
  ↓  
routing  
classification  
query decomposition  
memory importance  
reranking  
simple tool decisions  
  
2B-4B model  
  ↓  
reasoning  
planning  
code generation  
diagnosis  
complex tool use
```

The smallest possible model should perform each task that can be reliably delegated.

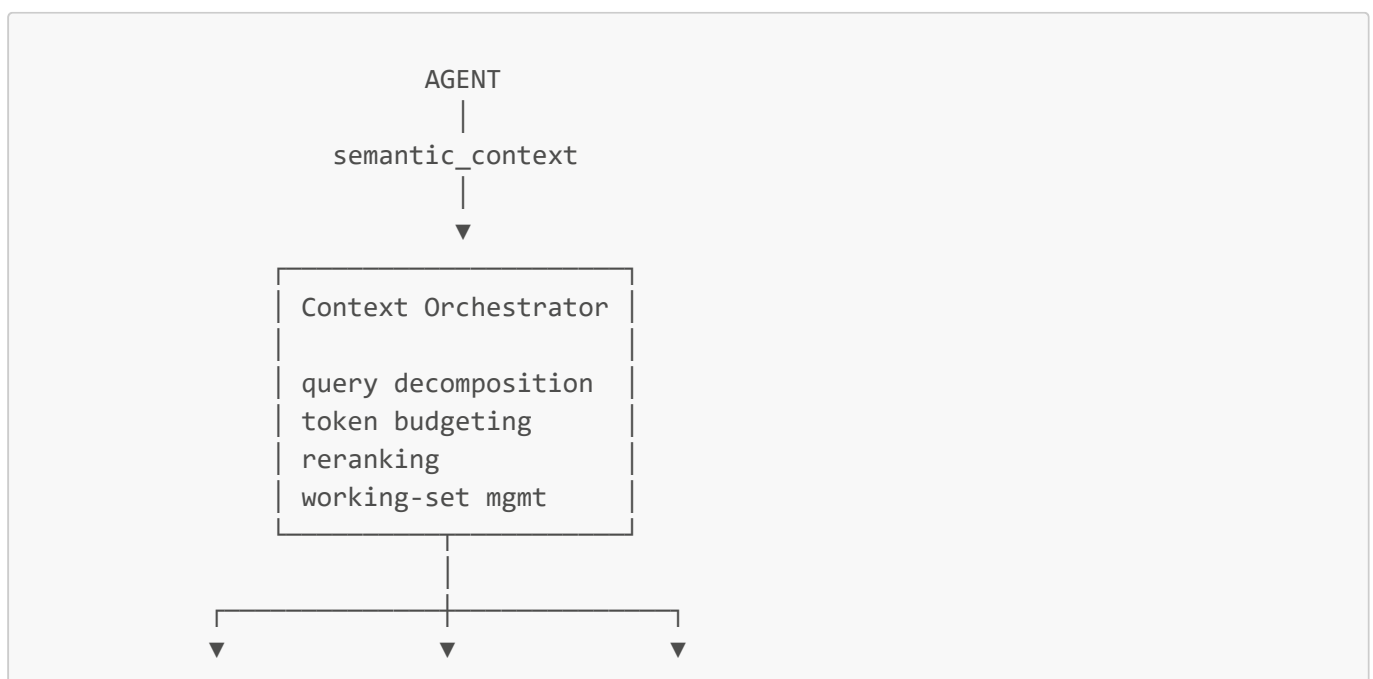
Deterministic systems should handle everything that does not require inference.

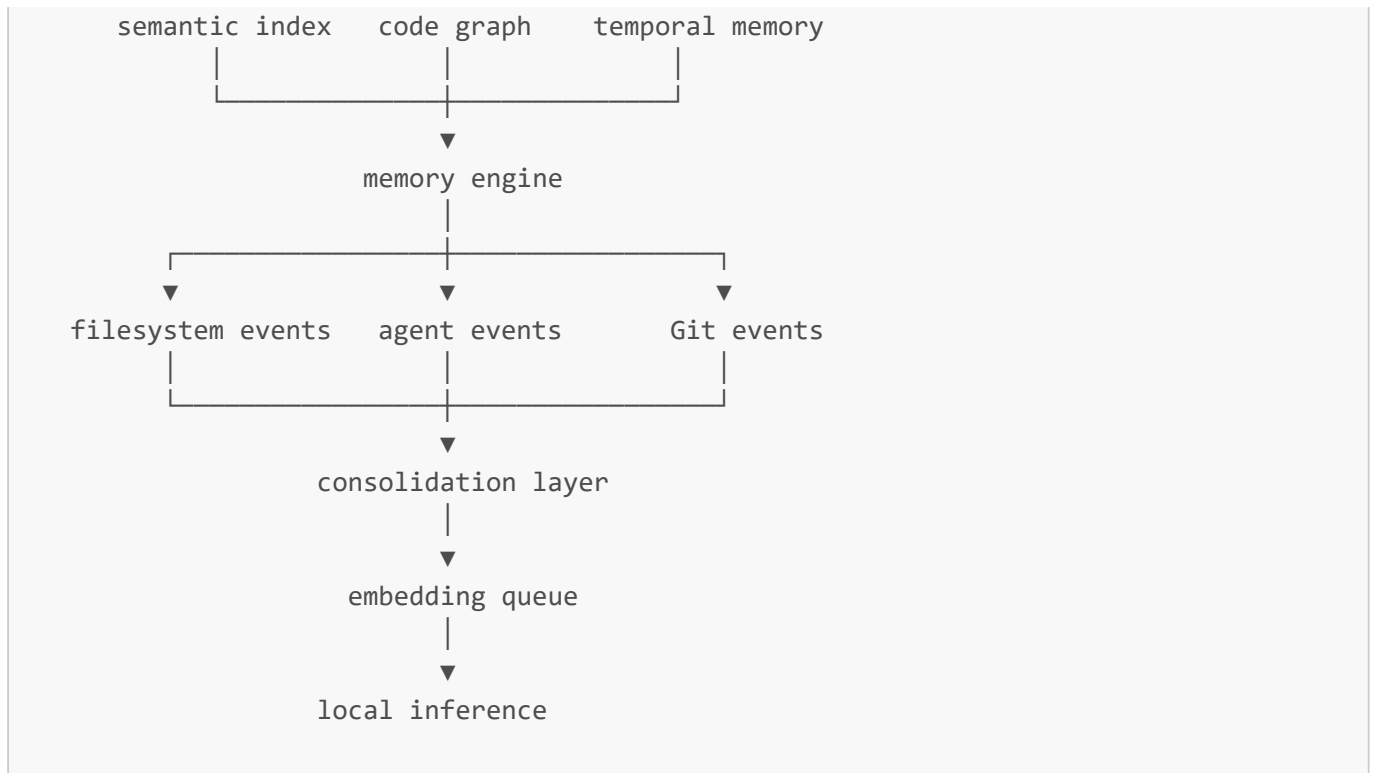
40. Mature Architecture

The eventual system might resemble:



A more advanced memory architecture becomes:





41. Observability

The system needs to explain itself.

Expose:

```
documents indexed
chunks indexed
pending jobs
failed jobs
last filesystem event
last successful index
embedding latency
search latency
index size
embedding model
stale chunks
working-set size
memory counts by type
```

CLI commands might include:

```
semantic-mcp status
semantic-mcp search "query"
semantic-mcp reindex
semantic-mcp doctor
```

doctor should verify:

```
embedding backend reachable?  
database writable?  
workspace accessible?  
watcher alive?  
MCP configuration valid?  
embedding dimensions consistent?  
Tree-sitter parsers available?  
index stale?
```

42. Evaluation

The project should be measured against **task outcomes**, not merely retrieval quality.

Build a fixed evaluation suite.

Questions might include:

```
Where is X initialized?  
  
Why does Y behave this way?  
  
Which function handles Z?  
  
What decision did we make about Q?  
  
What changed during the previous session?  
  
What caused the last compiler failure?  
  
Which files are relevant to this bug?
```

Retrieval metrics:

```
recall@k  
MRR  
latency  
precision
```

System metrics:

```
embedding calls per edit  
chunks regenerated per edit
```

```
indexing latency
context tokens consumed
time to recover session state
```

Agent metrics:

```
task success rate
iterations to success
wall-clock time
inference tokens
energy consumption
VRAM requirements
failure recovery rate
```

One particularly important measurement is:

How much model context can semantic retrieval replace without reducing task success?

And ultimately:

Given the same software-engineering task, can the small-model system eventually produce an equally correct artifact as a frontier model?

43. Recommended Vertical Milestones

Development should proceed through usable vertical slices rather than isolated subsystem completion.

Milestone A: Live Semantic Code Search

```
run service on project
  ↓
repository indexed
  ↓
edit file
  ↓
watcher detects change
  ↓
affected chunks re-embedded
  ↓
agent queries MCP
  ↓
fresh implementation returned
```

This alone is already useful.

Milestone B: Persistent Session Memory

```
agent makes decision
  ↓
decision recorded
  ↓
session ends
  ↓
new context begins
  ↓
agent retrieves decision
  ↓
work continues
```

This establishes continuity.

Milestone C: Automatic Context Resurrection

```
new session
  ↓
resume_context
  ↓
task + decisions + failures
+ relevant files + Git state
  ↓
agent continues autonomously
```

Milestone D: Experience-Based Recovery

```
task fails
  ↓
failure retrieved against past failures
  ↓
previous successful fix discovered
  ↓
model adapts solution
  ↓
deterministic verification
```

At this point the system begins accumulating capability through experience.

44. Research Thesis

The larger research question behind the project is:

How much of frontier-model performance can be reconstructed through systems engineering rather than parameter count?

The hypothesis is that many capabilities commonly purchased through model scale can instead be partially externalized:

```
memory
→ persistent retrieval

long context
→ dynamic context construction

type reasoning
→ compiler

exact knowledge
→ deterministic tools

task persistence
→ explicit state

self-correction
→ verification loops

experience
→ semantic failure/fix memory

code understanding
→ parsers + code graphs

attention
→ working-set management

long-horizon reasoning
→ iterative stateful execution
```

This will not make a small model universally equivalent to a frontier model.

Some tasks depend fundamentally on abstraction ability, broad world knowledge, or reasoning capacity that may remain beyond a small model's capability ceiling.

But agentic work inside a well-instrumented environment is different.

Software development in particular provides enormous amounts of deterministic feedback.

The system can therefore trade:

```
model size
```

for some combination of:

```
time  
iteration  
memory  
retrieval  
tooling  
verification  
deterministic computation
```

45. Guiding Philosophy

The system should optimize for:

eventual correctness rather than one-shot brilliance

and:

use inference only where inference adds value

Everything else should be delegated to cheaper, more reliable mechanisms whenever possible.

The model does not need to remember everything.

The memory system remembers.

The model does not need to know whether code compiles.

The compiler knows.

The model does not need to remember repository structure.

The filesystem and parser know.

The model does not need to carry the entire history of a project.

The semantic system reconstructs the relevant history when needed.

The model does not need to solve a complex problem perfectly on its first attempt.

The controller allows it to investigate, fail, observe, revise, and try again.

46. End State

The eventual goal is not merely:

a semantic embeddings MCP server

Nor merely:

RAG for source code

Nor even:

persistent agent memory

The larger system is:

A persistent, temporal, structured cognitive substrate for small local agents, designed to substitute systems engineering, deterministic computation, external memory, verification, and iterative execution for capabilities that would otherwise require substantially larger models.

The semantic-memory service is one foundational component of that architecture.

Its most important product is not embeddings.

It is **continuity**.

And the larger project's most important resource is not parameter count.

It is the ability to make a small model operate as part of a system that can **remember, inspect, verify, recover, accumulate experience, and keep trying**.